Smart Commerce

An AI-Driven Smart E-Commerce Platform with Role-Based Dashboards, Inventory Intelligence, and Demand Forecasting

Final Graduate Project Report

Prepared by:

Student Name: .....

Student ID: .....

Department of Computer Science and Engineering

University: .....

Supervised by

Supervisor Name: .....

Designation: .....

June 2026

Attestation

I am aware that plagiarism is against academic rules and regulations and is completely forbidden. I confirm that this project report has been prepared for the final project work of Smart Commerce. The implementation, analysis, and documentation are based on the developed project repository, and any external technologies, libraries, frameworks, and referenced concepts have been acknowledged properly.

Author Name: ..... Signature: .....

Evaluation Committee

Role	Name	Signature
Supervisor		
Internal Examiner 1		
Internal Examiner 2		
External Examiner		

Declaration

This is to declare that the work on "Smart Commerce: An AI-Driven Smart E-Commerce Platform with Role-Based Dashboards, Inventory Intelligence, and Demand Forecasting" is an original final project developed as part of the academic requirement for a graduate software engineering project. The project is not a direct copy of any existing system. The system was designed and implemented as a full-stack web application using a modern frontend, backend API, database layer, authentication model, administrative workflow, vendor workflow, customer workflow, and a Python-based demand forecasting component.

Approved by: .....

Supervisor

Acknowledgement

I would like to express sincere gratitude to my respected supervisor, teachers, and department for their guidance, support, and valuable academic direction throughout the completion of this final project. Their advice helped shape the project into a practical software solution that combines e-commerce operations with analytics, inventory control, and intelligent forecasting. I am also thankful to my classmates, friends, and family members for their encouragement during the design, development, testing, and documentation stages of the work.

Abstract

The rapid growth of online commerce has created a strong need for platforms that can support not only product selling, but also operational control, inventory planning, order tracking, analytics, and decision support. Many small and medium businesses depend on separate tools for product listing, stock monitoring, customer communication, sales analysis, and forecasting. This separation increases operational effort and reduces the ability to make timely business decisions. This project presents Smart Commerce, a full-stack smart e-commerce platform developed to integrate customer shopping, vendor product management, administrator control, inventory monitoring, audit activity, chatbot assistance, PDF order receipts, and demand forecasting within a single web-based system.

The platform is implemented with Next.js, React, TypeScript, Tailwind CSS, Express.js, Node.js, MongoDB, Mongoose, JWT authentication, Docker, Recharts, jsPDF, and a Python forecasting script. The system supports three primary operational users: customers, vendors, and administrators. Customers can browse products, search and filter the catalog, add items to cart, complete checkout, download receipts, and track order progress. Vendors can manage products, inventory, and order status. Administrators can observe key performance indicators, manage users and vendors, monitor low-stock products, view audit logs, control orders, and review future demand predictions. The demand forecasting module uses historical order movement, weighted averages, trend detection, keyword-based product signals, and confidence scoring to estimate short-term product demand.

The result is a practical commerce platform that demonstrates how a modern web architecture can be extended with analytics and intelligent inventory support. The project is academically valuable because it combines software engineering, database modeling, role-based access control, business process automation, and baseline predictive analysis in a single integrated system.

Table of Contents

1. Introduction

1.1 Project Overview

1.2 Aim

1.3 Scope of the Work

1.4 Research Questions and Motivation

2. Literature Review

2.1 Related Academic Foundations

2.2 Review of Existing Commercial and AI-Enabled Solutions

2.3 Research Gap

2.4 Justification of the Proposed System

3. System Description

3.1 System Perspective

3.2 System Users

3.3 Product Features
3.4 User Stories
4. Requirement Analysis
4.1 Functional Requirements
4.2 Non-Functional Requirements
4.3 System Requirements
4.4 SWOT Analysis
4.5 Use Case Analysis
4.6 Data and Business Rule Analysis
4.7 Formal Analysis and Modeling Artifacts
5. System Design and Implementation
5.1 System Architecture
5.1.1 Research Methodology and Project Approach
5.1.2 Database Design
5.1.3 API and Route Design
5.2 Core Functional Modules
5.3 Order and Forecasting Workflow
5.4 Implementation
5.5 Security Design
5.6 Forecasting Algorithm Design
5.6.1 AI Model Comparison and Evaluation Metrics
5.7 Testing and Validation Strategy
6. Result Analysis
6.1 Practical Implementation
6.2 Quantitative Performance and Effectiveness Analysis
6.3 Forecasting Accuracy and Model Evaluation
6.4 User Acceptance and Usability Evaluation
6.5 Comparison with Traditional E-Commerce Systems
6.6 Research Contribution, Novelty, and Practical Implications
6.7 Expanded Limitations
7. Summary and Conclusion
7.1 Future Work Roadmap
7.2 Conclusion
References

1. Introduction

1.1 Project Overview

Smart Commerce is a web-based e-commerce and business operations platform designed to support marketplace-style product selling, order management, inventory supervision, analytics, auditability, and demand forecasting. The system was developed as a full-stack project with a clear separation between frontend user experience, backend business logic, database persistence, and forecasting computation. It uses a modern Next.js frontend for storefront and dashboard interfaces, and an Express.js backend for secure API access, data validation, authentication, authorization, and aggregation of business data.

The platform is different from a simple online shop because it focuses on operational intelligence. It provides customer-facing pages for browsing, cart, checkout, and order tracking, but it also includes administrative modules for monitoring revenue, recent orders, low-stock products, audit activity, top products, users, vendors, inventory, and demand forecast results. A vendor can add products, manage product status, update stock, and track orders. This multi-role design makes the platform suitable for small and medium businesses that need both sales functions and internal control.

1.2 Aim

The main aim of the project is to design and implement a smart e-commerce platform that combines online selling with role-based management, inventory intelligence, business analytics, and baseline AI-driven demand forecasting. The project attempts to demonstrate how a practical commerce system can help business users make faster and more informed decisions while maintaining a clean and usable experience for customers.

- To build a modern storefront where customers can discover, filter, purchase, and track products.
- To implement secure authentication and role-based access for administrators, vendors, and customers.
- To create admin and vendor dashboards that support daily business operations.
- To develop inventory monitoring with low-stock alerts and stock adjustment logs.
- To integrate a Python-based demand forecasting module using previous order history.
- To provide audit logs and structured reporting features for governance and accountability.

1.3 Scope of the Work

The project scope includes the design and implementation of a complete full-stack application. The implemented modules cover storefront commerce, customer account flow, vendor operations, administrator management, analytics overview, inventory control, demand prediction, and a rule-based chatbot. The system uses seeded marketplace data that includes roles, categories, products, orders, forecast history, and audit logs. The scope is suitable for academic demonstration and can be extended toward production deployment with additional payment gateway integration and advanced machine learning models.

1.3.1 Storefront and Product Discovery

The storefront provides a public shopping interface with home page, product listing, product details, category-based filtering, search input, product cards, cart management, checkout, and customer login. The catalog includes grocery essentials, home care, personal care, kitchen and dining, electronics and gadgets, and fashion and lifestyle products.

1.3.2 Role-Based Dashboards

The system provides separate dashboard experiences for administrators, vendors, and customers. These dashboards are protected by session-aware routing and JWT-backed authentication. Administrative pages include orders, users, vendors, inventory, low-stock products, demand forecast, and catalog management. Vendor pages include overview, product management, inventory adjustment, and order supervision.

1.3.3 Inventory and Forecasting

The inventory module maintains product stock, reorder points, and inventory movement logs. The demand forecasting module reads historical order movement, groups it into six weekly buckets, and applies a Python script to generate predicted unit demand for the next 7, 14, and 21 days.

1.3.4 Reporting and Governance

The platform records audit activities for login attempts and sensitive administrative events. It also supports PDF receipt generation after checkout and report export for forecast data. These features strengthen traceability and provide usable output for business and academic evaluation.

1.4 Research Questions and Motivation

The motivation of this project comes from the observation that many commerce systems focus mainly on product listing and payment, while smaller businesses also need stock visibility, operational dashboards, and prediction support. Smart Commerce explores whether these needs can be integrated into one accessible platform.

How can a single platform support customers, vendors, and administrators without mixing their responsibilities?

How can inventory data and previous sales history be used to provide practical demand forecasting?

How can audit logs and role-based access improve governance in an e-commerce environment?

How can a modern full-stack architecture be organized so that future modules can be added with minimum disruption?

1.5 Problem Statement

Small and medium businesses often begin their online sales operation with a basic catalog or social media page. As the number of products, customers, vendors, and orders grows, the business gradually requires product control, stock visibility, order status management, analytics, and decision support. When these functions are handled manually or through disconnected tools, data becomes duplicated and business decisions become slower. A product may become out of stock before the administrator notices it, a vendor may not have a clear view of product status, and management may not understand which items are likely to sell in the near future.

The problem addressed by Smart Commerce is therefore not only how to sell products online, but how to manage the operational intelligence around online selling. The project attempts to reduce fragmentation by placing the customer storefront, vendor operations, administrative monitoring, inventory control, audit logging, and demand forecasting inside one application. This makes the project suitable as a software engineering solution because it combines business process design, data modeling, secure access control, and predictive analytics.

1.6 Project Objectives

The objectives of Smart Commerce were defined around both software functionality and business usefulness. The project is expected to deliver a working commerce platform while also demonstrating how operational data can support better decision-making.

- Develop a complete shopping flow where customers can browse products, add items to cart, checkout, and view order details.

- Create role-specific dashboards so that administrators, vendors, and customers receive interfaces according to their responsibilities.

- Improve inventory visibility by showing low-stock and out-of-stock items clearly to business users.

- Support business intelligence through KPIs, top products, recent orders, revenue trends, and forecast ranking.

- Add predictive support by estimating future product demand from previous order history.

- Maintain governance through login activity and selected audit records for sensitive operations.
- Prepare the system for reproducible demonstration through Docker setup, environment variables, and database seeding.

2. Literature Review

2.1 Related Academic Foundations

Modern e-commerce research shows that online selling systems are no longer limited to product display and checkout. They increasingly combine transaction processing, customer experience, analytics, recommendation logic, inventory visibility, and operational decision support. Schafer, Konstan, and Riedl's early work on e-commerce recommendation applications explains that product discovery becomes difficult when catalogs grow, and that recommendation services can reduce this search burden by connecting user behavior with relevant products. Later recommender-system research, including matrix factorization and neural collaborative filtering, shows how commercial platforms can use implicit behavior, purchase history, and user-item interactions to personalize the shopping experience. However, these methods normally require large behavioral datasets, careful evaluation, and safeguards against cold-start problems, bias, and limited explainability.

Inventory and demand forecasting are also central themes in supply chain and retail literature. Croston's work on intermittent demand, Syntetos and Boylan's studies on demand estimation, and later reviews of supply-chain forecasting emphasize that inventory decisions depend not only on current stock but also on the expected timing and volume of future demand. Fildes, Ma, and Kolassa argue that retail forecasting must bridge statistical modeling and practical store-level decisions, because an accurate forecast has limited value if it is not connected to replenishment, stockout prevention, and operational action. This is directly relevant to Smart Commerce because the platform attempts to connect order history, stock levels, low-stock alerts, and forecast ranking in one administrative workflow.

The literature also shows a tension between advanced artificial intelligence and practical software engineering. Advanced methods such as ARIMA, gradient boosting, recurrent neural networks, and hybrid recommendation models can improve accuracy when sufficient data is available, but they also increase implementation cost, data requirements, maintenance complexity, and evaluation burden. For a graduate software engineering prototype, an explainable baseline forecasting method is academically defensible when it is implemented as part of a complete system and when its limitations are acknowledged. Smart Commerce therefore uses weighted historical sales, trend labels, keyword signals, and confidence scoring as a transparent starting point rather than claiming the accuracy of a production-scale machine learning pipeline.

2.2 Review of Existing Commercial and AI-Enabled Solutions

Commercial platforms provide strong evidence that the core e-commerce problem is already well understood. Shopify supports inventory tracking, stock adjustment history, inventory reports, sell-through rate, ABC analysis, and days-of-inventory calculations. Adobe Commerce, previously associated with Magento, provides stronger enterprise inventory capabilities such as multi-source inventory, salable quantity calculation, reservations, and shipment matching. WooCommerce provides flexible product management and analytics inside the WordPress ecosystem, while Salesforce B2C Commerce Einstein provides recommendation services that collect shopper activity and return personalized recommendations. These systems are powerful, but they often separate advanced intelligence, automation, and governance into paid plans, marketplace extensions, enterprise modules, or external analytics services.

Platform / Solution	Core Strengths	Analytics or AI Support	Limitations Observed	Implication for Smart Commerce
Shopify	Hosted storefront, product catalog, checkout, inventory tracking, reports, app ecosystem	Inventory reports, sell-through rate, ABC analysis, days of inventory remaining, automation through Flow and apps	Advanced forecasting, multi-vendor governance, and specialized low-stock workflows often depend on plan level, apps, or custom configuration	Shows the value of usable inventory reports, but leaves room for a compact academic prototype with built-in forecasting and role-based dashboards

Adobe Commerce / Magento	Enterprise catalog, configurable products, multi-source inventory, salable quantity, reservations, shipment logic	Strong reporting and extensible architecture; advanced intelligence normally requires enterprise services, BI tools, or extensions	High setup complexity for small teams; forecasting and audit-centered decision support are not simple default workflows	Supports the need for modular architecture while motivating a lighter system focused on administrator and vendor usability
WooCommerce	WordPress integration, product management, flexible themes and extensions, accessible setup for small stores	Built-in analytics and sales reports with date ranges, filters, CSV export, and dashboard metrics	Advanced forecasting, structured RBAC dashboards, audit logs, and marketplace-style vendor operations usually require plugins or custom development	Demonstrates SME accessibility, but highlights fragmentation when intelligence and governance are added through separate extensions
Salesforce Commerce Cloud Einstein	Enterprise commerce personalization and recommendation APIs connected to shopper activity	AI recommendations based on user events, recommender zones, reporting, and scalable service infrastructure	Primarily enterprise-oriented; recommendation intelligence is strong, but inventory forecasting and transparent academic model inspection are not the main focus	Useful benchmark for AI-enabled commerce, while Smart Commerce focuses on explainable operational intelligence for a smaller prototype
Custom MERN / Next.js commerce systems	Full control over frontend, backend, database models, roles, APIs, and deployment	Analytics and forecasting can be designed around project-specific data and workflows	Requires more engineering effort, testing, security hardening, and maintenance than hosted platforms	Justifies building Smart Commerce as a software engineering artifact that integrates commerce, governance, analytics, and forecasting end to end

The comparison indicates that existing platforms either provide strong selling infrastructure with intelligence added through extensions, or provide advanced enterprise AI that is costly and less transparent for a student-built prototype. This does not reduce the value of existing systems; instead, it clarifies the specific academic space for Smart Commerce: a smaller but integrated implementation where every major workflow can be inspected, modified, tested, and explained.

2.3 Research Gap

The main research gap is the limited availability of an integrated, explainable, and role-aware e-commerce prototype that combines storefront operations, vendor management, administrator governance, inventory monitoring, audit activity, and demand forecasting within one coherent software artifact. Academic forecasting studies often focus on model performance but do not implement the complete commerce workflow around the forecast. Commercial platforms implement the workflow well but may hide forecasting and intelligent decision support behind external applications, paid services, or enterprise-level configuration. As a result, small and medium businesses, students, and evaluators may struggle to see how data moves from customer orders into inventory decisions and forecast outputs.

A second gap concerns explainability and actionability. Many AI-enabled commerce systems emphasize personalization, recommendation, or automated optimization, but operational users still need understandable indicators such as recent sales, low-stock status, trend direction, forecast horizon, confidence, and vendor responsibility. Without these indicators, a prediction can become a black-box output rather than a useful management tool. Smart Commerce addresses this by making the forecast visible beside product names, vendors, stock information, weekly history, trend labels, and exportable reports.

A third gap is governance. In many small commerce implementations, audit logs, role-based permissions, inventory changes, and forecast results are treated as separate concerns. Smart Commerce treats them as related software engineering requirements. Administrators need visibility over users, vendors, orders, inventory risks, and forecast outputs; vendors need product and stock workflows; customers need a reliable shopping and tracking experience. This role separation creates a clearer operational model and makes the project stronger than a simple CRUD-based online shop.

2.4 Justification of the Proposed System

Based on the reviewed literature and platform comparison, Smart Commerce is justified as a graduate-level software engineering project because it demonstrates an end-to-end integration problem rather than only a single algorithm or a simple storefront. The project contributes a working architecture where customer transactions generate operational data, operational data informs dashboards and inventory alerts, and historical sales data is transformed into future demand estimates. The current forecasting model is intentionally baseline and explainable, which makes it suitable for demonstration, testing, and future comparison against advanced models such as exponential smoothing, ARIMA, Prophet, random forest, gradient boosting, or neural sequence models.

3. System Description

3.1 System Perspective

Smart Commerce is designed as a modular client-server web application. The frontend is responsible for rendering the storefront and role-based dashboards. The backend is responsible for authentication, authorization, data validation, business logic, database access, analytics aggregation, order processing, inventory adjustment, and forecast orchestration. MongoDB Atlas is used as the operational database, and Docker Compose is provided for containerized deployment of the frontend and backend services.

Layer	Technology Used	Purpose
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS, shadcn-style UI	Storefront, dashboards, forms, charts, and client workflows
Backend	Node.js 22, Express.js 5, TypeScript	REST API, validation, business services, authentication, and authorization
Database	MongoDB Atlas with Mongoose	Persistent storage for users, roles, products, categories, orders, inventory logs, and audit logs
Analytics and Forecast	Recharts, Python script, weighted/trend logic	Dashboard charts and demand prediction
Security	JWT, httpOnly cookies, bcryptjs, helmet, CORS, rate limiting	Session protection, password hashing, and secure API behavior
Deployment	Docker and Docker Compose	Repeatable local and containerized execution

3.2 System Users

User Type	Main Responsibilities
Customer	Browse products, search/filter catalog, add to cart, checkout, download receipt, and track orders.
Vendor	Create and manage products, update product status, adjust stock, and monitor vendor-side orders.
Administrator	View KPIs, manage users and vendors, control orders, monitor inventory, review audit logs, and inspect demand forecasts.
System	Generate tracking numbers, calculate totals, record audit logs, prepare forecast inputs, and execute Python prediction logic.

3.2.1 User Access Matrix

A major design decision in Smart Commerce is the separation of access according to business responsibility. Instead of presenting the same interface to all users, the system divides features by role. This improves usability because a customer does not see administrative controls, and it improves security because operational functions require authenticated and authorized access.

- Customers focus on product browsing, cart, checkout, personal order history, receipt download, and order tracking.

- Vendors focus on product creation, product status control, inventory adjustment, and order handling.
- Administrators have the broadest access, including users, vendors, orders, inventory, analytics, audit logs, and demand forecasting.
- System-level operations such as order number generation, shipment updates, and forecast execution are performed automatically by backend services.

Figure 3.1: Role-based access overview for the main Smart Commerce feature areas.

3.3 Product Features

3.3.1 Product Catalog

The product catalog supports product name, slug, SKU, description, category, vendor, price, compare-at price, stock, reorder point, rating, status, featured flag, tags, images, and product metrics. Products can be active, draft, or archived.

3.3.2 Cart and Checkout

Customers can add products to cart, review item quantities, enter delivery address, choose cash on delivery, card, or bank payment, and submit an order. The checkout computes subtotal, shipping fee, and total amount.

3.3.3 Order Tracking

Orders are created with unique order numbers and statuses such as pending, processing, shipped, delivered, and cancelled. Shipment details include carrier, tracking number, estimated delivery, and current location.

3.3.4 PDF Receipt Generation

After successful checkout, the frontend generates a PDF receipt using jsPDF. The receipt includes order number, customer details, delivery address, product lines, shipping fee, and final total.

3.3.5 Admin Analytics

The admin overview shows revenue, order count, active products, customer count, recent orders, inventory alerts, top products, and audit activity. Recharts is used for revenue and forecast visualization.

3.3.6 Low-Stock Alerts

The system identifies active products with stock below the configured low-stock threshold. It displays product name, vendor, SKU, current stock, threshold, urgency, and recommended reorder quantity.

3.3.7 Demand Forecasting

The forecasting module uses six weeks of order history, calls a Python script, and returns predicted units, recent sold quantity, trend label, keywords, confidence, and future demand for 7, 14, and 21 days.

3.3.8 Chatbot Assistance

The frontend includes a rule-based chatbot knowledge base that can answer questions about project features, product categories, order flow, demo accounts, dashboard modules, low stock, and demand forecasting.

3.3.9 Audit Activity

Audit logs store actor, actor email, action, entity type, entity id, status, IP address, metadata, and timestamp. The login service records successful and failed login attempts.

3.4 User Stories

- As a customer, I want to browse available products and place an order without visiting a physical store.
- As a customer, I want to download a receipt after checkout so that I have proof of purchase.
- As a vendor, I want to add and manage products so that my items remain visible in the marketplace.
- As a vendor, I want to update inventory levels so that customers do not order unavailable products.
- As an administrator, I want to view dashboard metrics so that I can understand business performance quickly.
- As an administrator, I want to identify low-stock products so that restocking decisions can be made on time.
- As an administrator or analyst, I want demand forecast data so that future product movement can be estimated.

3.5 Dataset and Seeded Demonstration Data

For a final project, seeded data is important because it allows the system to be demonstrated without manual database preparation. Smart Commerce includes a seed script that creates the core roles, demo users, categories, products, orders, forecast-history orders, and audit log examples. This gives examiners and users an immediate view of how the system behaves in a realistic marketplace environment.

- Six roles are seeded: super_admin, admin, inventory_manager, analyst, vendor, and customer.
- Three demo users are created for immediate testing: Platform Administrator, Demo Vendor, and Demo Customer.
- Six categories are included: Grocery Essentials, Home Care, Personal Care, Kitchen and Dining, Electronics and Gadgets, and Fashion and Lifestyle.
- Fourteen products are seeded with price, stock, reorder point, tags, images, ratings, and sales metrics.
- Customer orders are seeded with different statuses such as pending, processing, shipped, and delivered.
- Forecast-history orders are included so the demand forecasting page can show meaningful weekly movement.
- Audit log examples are provided for product creation, inventory threshold adjustment, analytics view, and login activity.

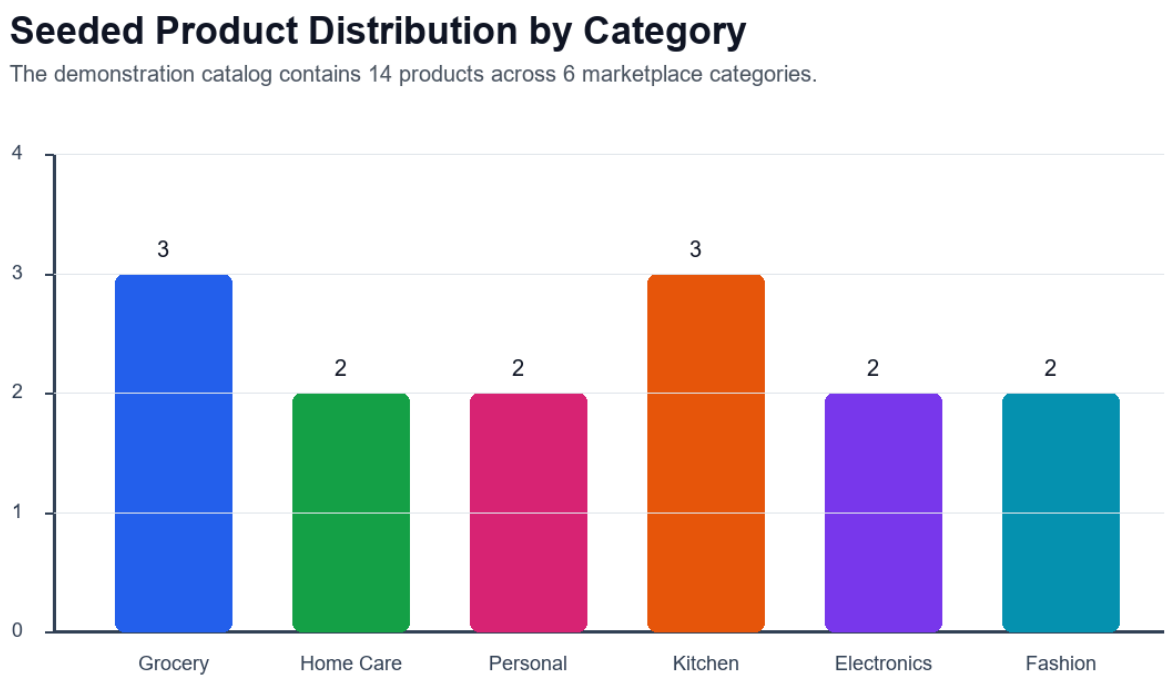

Figure 3.2: Seeded catalog distribution across the six product categories.

3.6 Business Rules

The system follows a number of practical business rules that make the application behave like a real commerce platform. Products must have valid categories before they can be created. Product stock cannot become negative after adjustment. Orders must contain existing products. Customers can see only their own orders, while administrators and vendors can view broader operational order lists. The order status is limited to defined stages so that tracking remains consistent.

- Shipping fee is calculated during checkout; orders above the configured amount can receive free shipping.
- Order status follows the path pending, processing, shipped, delivered, or cancelled.
- Shipment information is created when an order becomes shipped.
- Low-stock alerts are generated when active product stock is below the low-stock threshold.
- Inventory logs record stock before and after each adjustment.
- Demand forecast ignores cancelled orders and uses valid historical movement only.
- Disabled user accounts cannot log in even with a correct password.

4. Requirement Analysis

4.1 Functional Requirements

Requirement	Description
Authentication	The system shall authenticate users through email and password and issue JWT-based access tokens.
Role-Based Access	The system shall restrict dashboards and API functions according to user role.
Product Management	The system shall support product creation, update, listing, stock adjustment, and archival.

Catalog Browsing	The system shall allow customers to view featured products, product details, categories, and related products.
Order Placement	The system shall create orders from cart items and calculate subtotal, shipping fee, and total.
Order Status Update	The system shall allow administrators and vendors to update order status.
Inventory Logging	The system shall record stock changes with before and after values.
Dashboard Analytics	The system shall display KPIs, revenue series, recent orders, top products, and audit activity.
Demand Forecast	The system shall generate future product demand predictions from historical order records.
Receipt Export	The system shall generate a PDF receipt after checkout.
Forecast Export	The system shall provide report export support for demand forecast output.
Chatbot	The system shall answer common project and commerce questions using a local rule-based knowledge base.

4.2 Non-Functional Requirements

Quality Attribute	Requirement
Performance	Pages and APIs should respond quickly for standard academic demonstration data and should avoid unnecessary database work.
Security	Passwords must be hashed, JWT tokens must protect private routes, and account status must be checked during login.
Scalability	The modular backend and database schema should allow additional modules such as payment gateway, wishlist, and recommendation engine.
Maintainability	Code should be separated into models, routes, controllers, services, components, and pages.
Usability	Storefront and dashboards should be clear, responsive, and understandable for non-technical business users.
Reliability	The backend should use centralized error handling and fallback forecast logic if Python execution fails.
Data Management	Operational data should remain in MongoDB and should be accessed only through backend APIs.

4.3 System Requirements

Category	Requirement
Software	Node.js 22 or later, npm, Docker Desktop, MongoDB Atlas connection, modern browser, Python for forecasting script.
Frontend Runtime	Next.js development server on port 3000.
Backend Runtime	Express API server on port 5000.
Database	MongoDB Atlas database named smart_ecommerce.
Hardware	A standard development laptop or desktop with at least 8 GB RAM is suitable for local demonstration.

4.4 SWOT Analysis

Area	Analysis

Strengths	Integrated customer, vendor, and admin workflows; modern stack; forecasting support; inventory alerts; audit logging; PDF receipt generation.
Weaknesses	Forecasting is a baseline model, not a fully trained machine learning pipeline; online payment gateway is not fully integrated; test suite can be expanded.
Opportunities	Can be extended with real payment, recommendation engine, notification service, warehouse routing, advanced analytics, and mobile app support.
Threats	Real-world e-commerce systems require strong security hardening, payment compliance, privacy protection, and high availability under traffic spikes.

4.5 Use Case Analysis

Use case analysis helps explain how the main actors interact with the system to complete business tasks. Smart Commerce has three major human actors: customer, vendor, and administrator. The system actor also performs background tasks such as order number generation, tracking event creation, and forecast execution.

- Login: an active customer, vendor, or administrator account can create a JWT-backed session and enter the correct area of the platform.
- Browse products: a customer can inspect product price, rating, category, stock, and product details from the storefront.
- Place order: a logged-in customer can submit cart items, delivery address, and payment method to create an order and download a receipt.
- Update order status: a vendor or administrator can move an order through pending, processing, shipped, delivered, or cancelled states.
- Add product: a vendor or administrator can create a product when a valid category exists.
- Adjust stock: an authorized user can update stock while the system stores an inventory log with before and after values.
- Review dashboard: an administrator can view KPIs, charts, low-stock alerts, recent orders, top products, and audit activity.
- View demand forecast: an administrator or analyst can inspect product demand predictions generated from historical order movement.

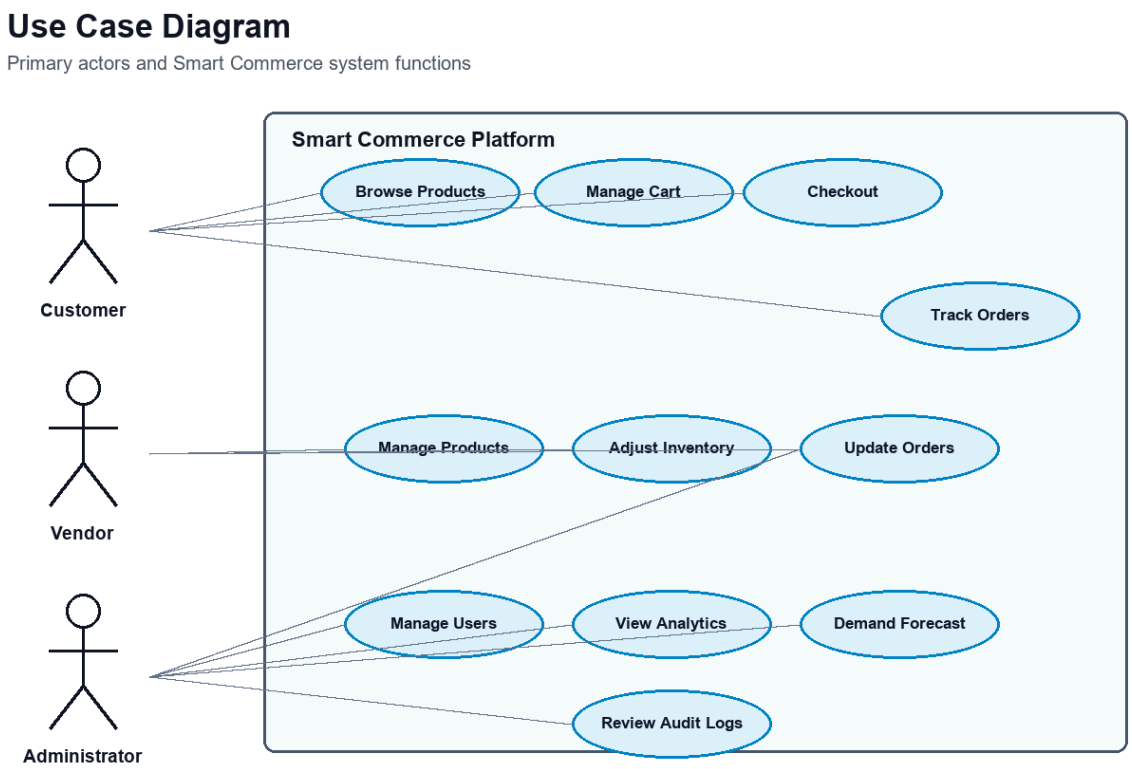

Figure 4.1: UML use case diagram showing customer, vendor, and administrator interactions.

4.6 Data and Business Rule Analysis

The main data entities of the project are users, roles, categories, products, orders, inventory logs, and audit logs. These entities represent both operational data and governance data. Operational data supports direct commerce workflows such as catalog browsing and order placement. Governance data records who performed important actions and whether the action was successful.

Entity	Important Fields	Purpose
User	name, email, passwordHash, role, status, lastLoginAt	Stores account identity and access state.
Role	name, description, permissions	Defines what type of access a user should have.
Category	name, slug, description	Groups products into customer-friendly catalog sections.
Product	name, slug, SKU, price, stock, reorderPoint, status, metrics	Stores sellable item and inventory information.
Order	orderNumber, customer, items, payment, total, shipment, trackingEvents	Stores customer purchase and delivery progress.
InventoryLog	product, quantity, stockBefore, stockAfter, reason, createdBy	Records stock movement history.
AuditLog	actor, action, entityType, status, ipAddress, metadata	Records security and governance activity.

Data validation is handled at multiple levels. Mongoose schemas define required fields, allowed status values, numeric minimums, and indexes. Backend services also check business conditions, such as whether a product exists before order creation or whether a category exists before product creation. This layered validation reduces the possibility of invalid operational data being saved.

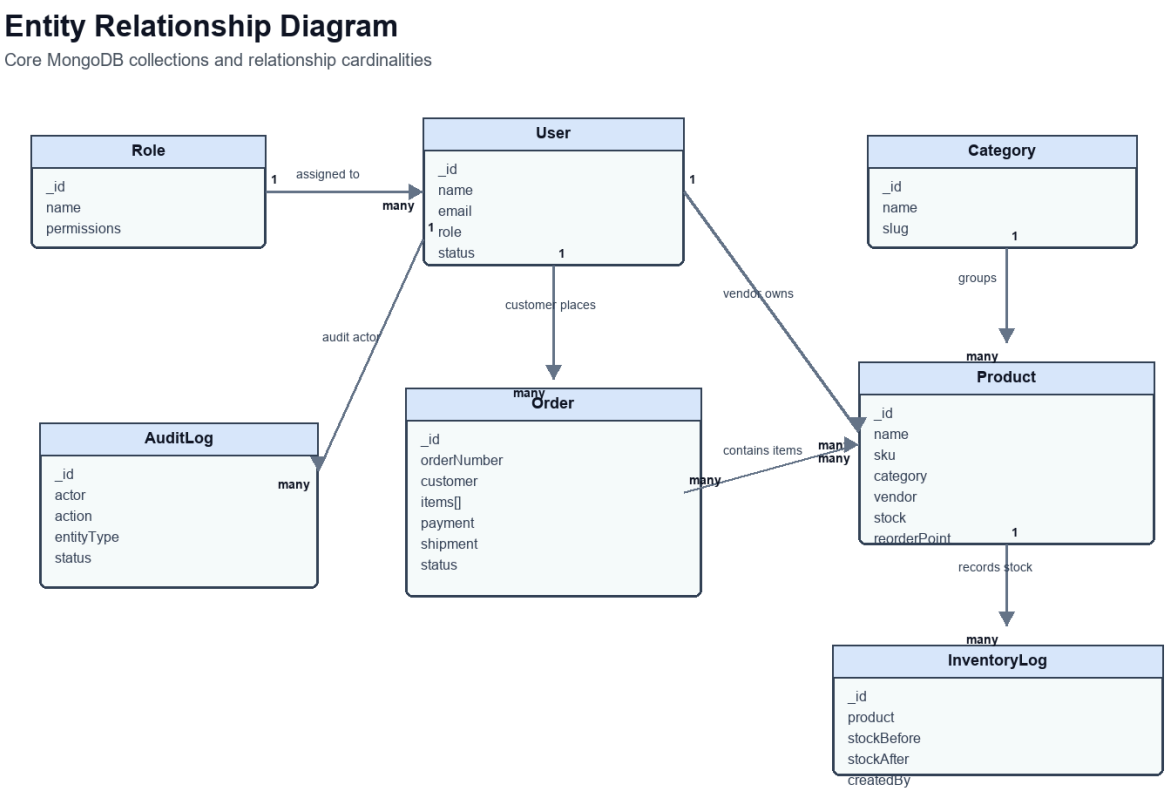

Figure 4.2: Entity relationship diagram for the main Smart Commerce collections.

4.7 Formal Analysis and Modeling Artifacts

To strengthen design rigor, the project uses formal software engineering artifacts in addition to textual descriptions. The use case diagram supports requirement traceability by connecting actors with system functions. The ERD explains persistent data relationships. The class diagram clarifies domain models and service responsibilities. The sequence diagram shows runtime interaction among users, frontend, backend, database, and forecast script. The activity diagram models the order and inventory workflow. The deployment diagram explains how the system is executed across browser, frontend service, backend service, database, and Python runtime.

Artifact	Purpose in This Project	Traceability Value
Use Case Diagram	Shows actor-system interactions for customer, vendor, and administrator workflows.	Links user stories to functional requirements.
Entity Relationship Diagram	Shows data entities and relationships among users, roles, products, orders, inventory logs, and audit logs.	Links database design to business rules and persistence needs.
Class Diagram	Shows domain models, important attributes, methods, and service responsibilities.	Links code structure to analysis objects.
Sequence Diagram	Shows order creation and forecast request flow across runtime components.	Links API behavior to frontend/backend interaction.
Activity Diagram	Shows customer checkout, stock validation, order creation, and analytics update flow.	Links workflow logic to validation scenarios.
Deployment Diagram	Shows runtime nodes and service communication.	Links architecture decisions to deployment and operational concerns.

5. System Design and Implementation

5.1 System Architecture

The architecture follows a modular full-stack design. The frontend is organized by route groups such as storefront, dashboard, admin, and vendor. Shared components are kept under the components directory, while API helpers, cart state, authentication helpers, chatbot knowledge, and commerce utilities are kept under the lib directory. The backend is organized into configuration, middleware, models, modules, routes, and utility layers. Each backend feature module contains route, controller, and service files, which improves readability and keeps business logic separate from HTTP request handling.

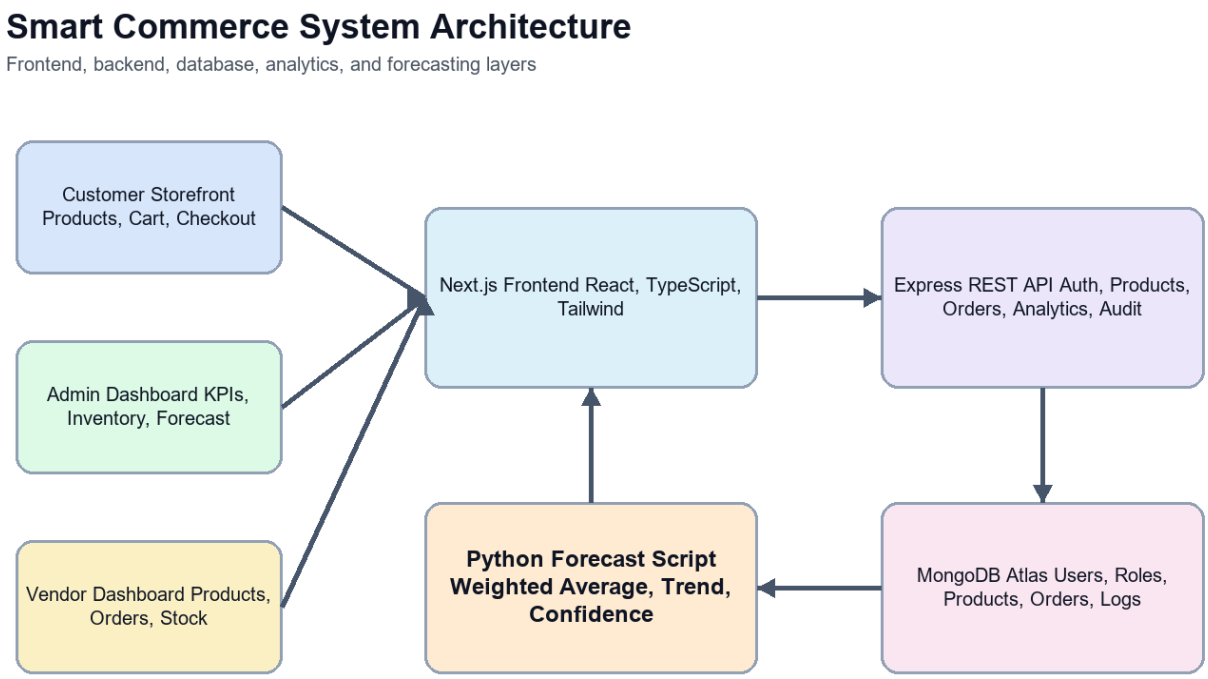

Figure 5.1: High-level Smart Commerce system architecture.

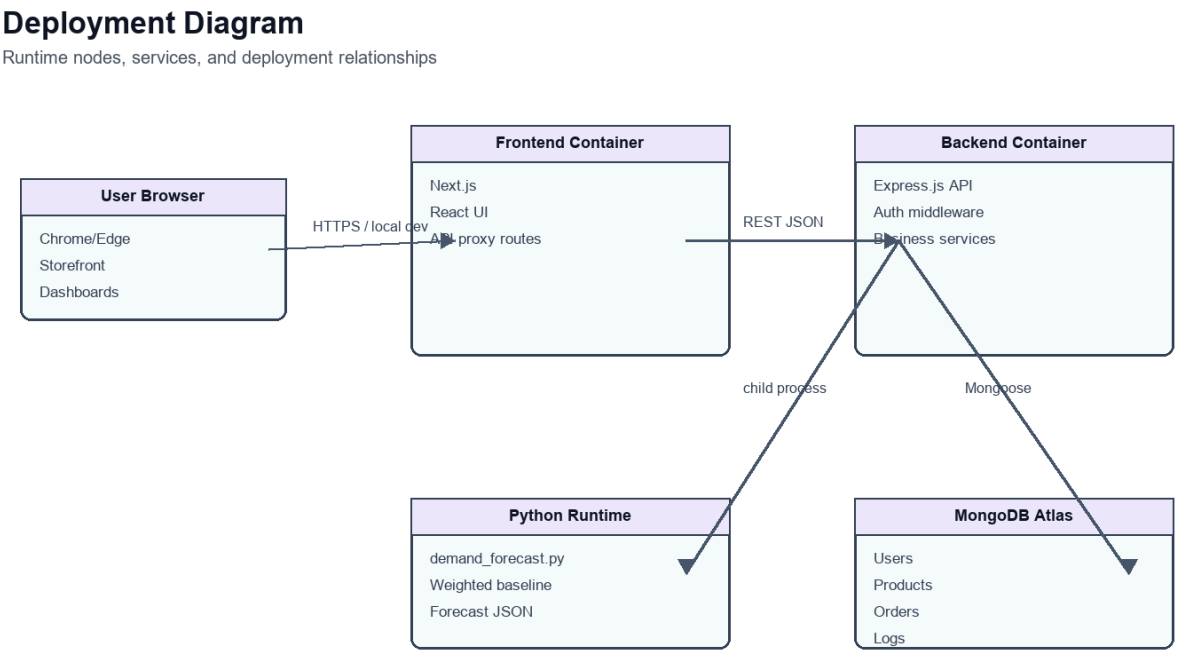

Figure 5.2: UML deployment diagram showing runtime services and infrastructure nodes.

Backend Module	Main Responsibility
auth	Login, current user lookup, JWT creation, password verification, and login audit logs.
products	Product listing, product details, product creation/update/archive, inventory overview, and stock adjustment.
orders	Order creation, order listing, order detail, order status update, tracking events, and shipment data.
analytics	Dashboard overview, low-stock alerts, top product data, revenue series, and demand forecast orchestration.
users	User and vendor management functions for administrative workflows.
audit	Audit log retrieval for governance and administrative visibility.
health	API health check route for service verification.

5.1.1 Research Methodology and Project Approach

This project followed a hybrid research and engineering methodology. Design Science Research (DSR) was used as the academic frame because the project creates and evaluates a purposeful software artifact for a practical business problem. DSR is appropriate for Smart Commerce because the main output is not only a written analysis, but a working e-commerce platform that embodies design decisions, business rules, data models, user workflows, and predictive decision support. Agile incremental development was used as the implementation approach because the system contains many connected modules that needed to evolve through small working increments.

The methodology therefore combined three layers: Design Science Research for problem framing and artifact evaluation, Agile-style iteration for building and refining modules, and a software engineering lifecycle structure for requirements analysis, architecture, implementation, verification, validation, and documentation. This combination allowed the project to be treated as a research-oriented engineering study rather than only a development narrative.

DSR Phase	Application in Smart Commerce	Evidence Produced
Problem identification and motivation	The project identified fragmentation in small and medium e-commerce operations, where storefront, vendor work, inventory monitoring, analytics, audit records, and demand forecasting are often separated.	Problem statement, research questions, literature review, platform comparison, and research gap.
Objectives of the solution	Objectives were converted into measurable system goals: role-based access, product catalog, cart and checkout, order tracking, inventory alerts, admin dashboards, vendor workflows, audit logging, and demand forecasting.	Functional requirements, non-functional requirements, user stories, and use case analysis.
Design and development	The artifact was designed as a modular full-stack system using Next.js, Express.js, MongoDB, JWT authentication, Mongoose models, Recharts dashboards, PDF export, and a Python forecasting script.	Architecture diagram, database design, API route design, implemented frontend and backend modules.
Demonstration	The system was demonstrated with seeded users, roles, products, categories, orders, inventory movement, forecast-history orders, and audit logs so that all key workflows could be executed without manual data preparation.	Seed script, demo accounts, storefront pages, admin dashboard, vendor dashboard, inventory pages, and demand forecast page.
Evaluation	The artifact was evaluated through requirement traceability, scenario-based validation, build/type checking, database seeding, security review, and inspection of dashboard and forecast outputs.	Testing and validation strategy, result analysis, implementation screenshots, build checks, and limitation analysis.
Communication	The project outcomes were documented for academic assessment and practical reuse.	Final report, README, generated charts, presentation materials, and source code organization.

Requirements were elicited from four sources. First, the project problem statement established the need for an intelligent e-commerce platform rather than a simple online shop. Second, the review of existing systems such as Shopify, Adobe Commerce, WooCommerce, and AI-enabled commerce services revealed common capabilities and limitations. Third, role analysis separated the needs of customers, vendors, and administrators. Fourth, implementation feasibility was considered by checking which features could be built and demonstrated within the selected technology stack. These sources were translated into user stories, functional requirements, non-functional requirements, database entities, API routes, and dashboard modules.

Requirement Source	Method Used	Resulting Design Decision
Literature and platform review	Compared academic forecasting/recommendation ideas with commercial e-commerce platform capabilities.	Include analytics, low-stock monitoring, forecast ranking, and a platform comparison in the report.
Actor and workflow analysis	Separated customer, vendor, administrator, and system responsibilities.	Create role-specific dashboards, protected routes, and role-aware backend authorization.
Business process decomposition	Broke commerce operations into product, cart, checkout, order, stock, analytics, audit, and forecast workflows.	Organize backend code into feature modules and frontend code into route groups and reusable components.
Data and validation needs	Identified persistent entities and important business constraints.	Use MongoDB collections for users, roles, categories, products, orders, inventory logs, and audit logs with Mongoose validation.
Demonstration requirement	Required a complete project that examiners could inspect without creating data manually.	Develop seed data covering users, products, categories, orders, forecast history, and audit examples.

Design decisions were made using modularity, traceability, security, and explainability as guiding criteria. A client-server architecture was selected to separate user interfaces from business logic. JWT with httpOnly cookies was selected to support protected sessions. Mongoose schemas were used to keep database validation close to the data model. The forecasting component was implemented as a Python script because Python is suitable for analytical computation and can be called from the backend service. A baseline explainable forecasting method was chosen instead of a complex machine learning model because the project needed transparent results that could be inspected and improved in future work.

Prototype Increment	Main Purpose	Validation Activity
Increment 1: Foundation	Create repository structure, frontend scaffold, backend scaffold, TypeScript configuration, MongoDB connection, Docker setup, and seed process.	Run project setup, verify database connection, and confirm seed data can be inserted.
Increment 2: Authentication and roles	Implement login, session retrieval, JWT cookie flow, account status checks, and protected admin/vendor/customer access.	Validate correct redirection, role separation, and failed-login behavior.
Increment 3: Commerce core	Build product catalog, product detail, cart, checkout, order creation, order confirmation, receipt generation, and order tracking.	Execute customer purchase scenarios and inspect saved order records.
Increment 4: Operations dashboards	Add admin KPIs, vendor product management, order status updates, inventory overview, stock adjustment logs, and audit activity.	Check dashboard metrics, inventory changes, status transitions, and audit log visibility.
Increment 5: Intelligence and reporting	Integrate low-stock alerts, demand forecasting, chart visualization, forecast ranking, chatbot support, and PDF forecast export.	Compare forecast output with seeded weekly order history and verify that export/report features work.

Validation was performed at both engineering and research levels. Engineering validation checked whether the system builds, routes respond, data is stored correctly, and protected workflows behave as expected. Research validation checked whether the artifact addresses the research gap identified in the literature review: integration of commerce, role-based operations, inventory intelligence, auditability, and explainable demand forecasting. The evaluation did not claim production readiness; instead, it assessed whether the implemented artifact is coherent, demonstrable, and aligned with graduate-level software engineering objectives.

Evaluation Aspect	Validation Method	Acceptance Evidence
Requirement coverage	Mapped implemented modules against functional requirements and user stories.	Storefront, cart, checkout, dashboards, inventory, orders, audit, and forecast modules are present.
Technical correctness	Used TypeScript-oriented build checks, backend compilation, route inspection, and seed execution.	Frontend/backend code compiles and seeded data supports repeatable demonstration.
Workflow validity	Ran scenario-based checks for customer purchase, admin monitoring, vendor inventory update, and forecast review.	Each actor can complete the expected workflow through role-specific pages.
Data validity	Checked Mongoose schemas, required fields, status values, stock quantities, and historical order buckets.	Invalid or incomplete data is reduced through schema and service-level validation.
Forecast usefulness	Compared generated predictions with recent and total sales history.	Forecast output includes predicted units, recent sold quantity, trend, confidence, and future horizons.
Research contribution	Compared final artifact against the research gap and commercial platform matrix.	The project demonstrates a transparent integrated prototype rather than a disconnected set of tools.

5.1.2 Database Design

The database design uses MongoDB collections with Mongoose schemas. This document-oriented approach is suitable because commerce records such as orders naturally contain nested item lists, shipping address, shipment data, and tracking events. At the same time, products, categories, users, and roles are modeled as separate collections so that they can be queried independently and referenced where required.

The Product collection is central to the platform. It includes selling information, merchandising information, and operational stock information. The Order collection stores a snapshot of ordered product names and prices, which is important because product names and prices may change later. Inventory logs and audit logs are separated from product and user documents because they represent historical events rather than current state.

Relationship	Design Explanation
User to Role	Each user references a role document, allowing role names and permissions to be managed centrally.
Product to Category	Each product references a category so catalog filters and groupings remain consistent.
Product to Vendor	Products may reference a vendor user, allowing vendor ownership to be displayed and managed.
Order to User	Orders may reference the customer user while also storing customer name and email.
Order to Product	Order items reference products but also store productName and unitPrice as purchase-time data.
InventoryLog to Product	Each stock movement references the affected product and records stock before and after.
AuditLog to User	Audit events may reference the actor while also storing actorEmail for traceability.

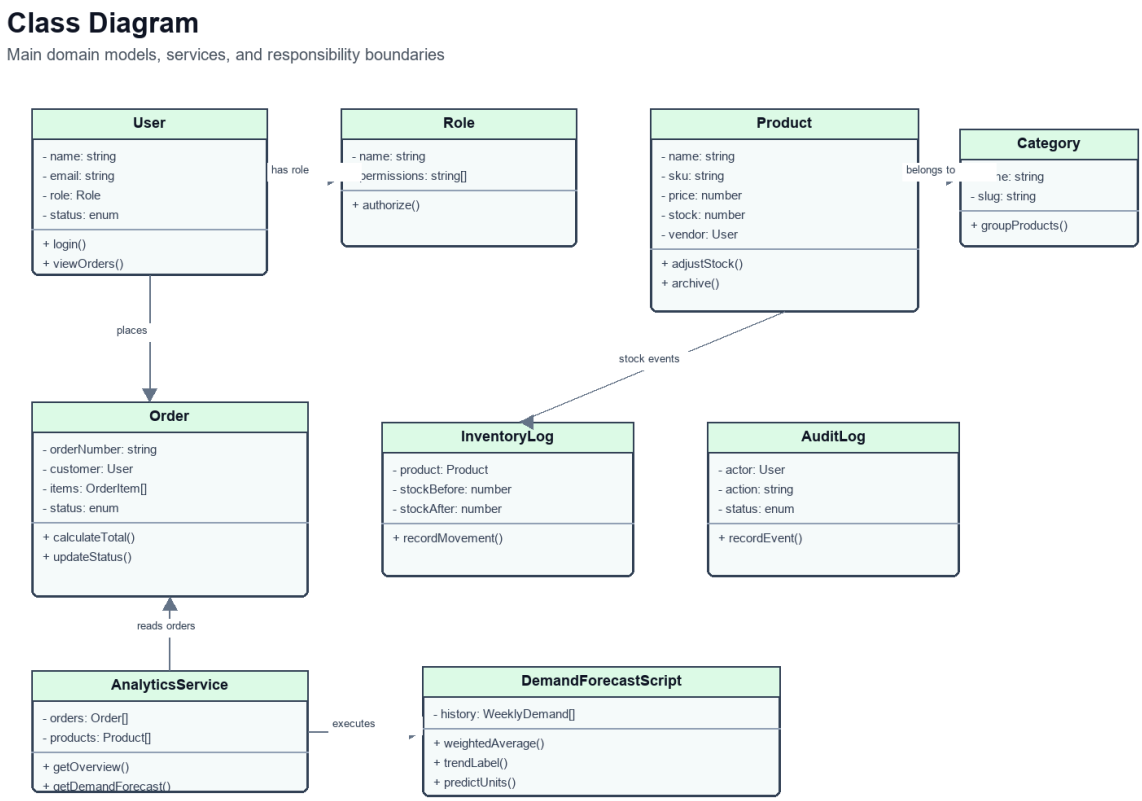

Figure 5.3: UML class diagram for the core Smart Commerce domain and analytics service.

5.1.3 API and Route Design

The backend API is grouped by domain. The root API router mounts health, authentication, products, orders, users, analytics, and audit routes. This organization allows each feature area to evolve without mixing unrelated logic. The frontend communicates with the backend through helper functions and Next.js API routes, which also help with session-aware behavior in the browser.

API Domain	Example Responsibility
/api/auth	Login, logout, session validation, and current user retrieval.
/api/products	Product listing, product details, product creation, update, inventory overview, and stock adjustment.
/api/orders	Order creation, order listing, order detail, and order status update.
/api/users	Customer and vendor management for administrative users.
/api/analytics	Dashboard overview, demand forecast, inventory alerts, and top product information.
/api/audit	Audit log listing for governance-aware administration.
/api/health	Backend service health verification.

5.2 Core Functional Modules

The project repository contains clearly separated functional modules. The customer-facing pages include home, products, product details, cart, checkout, login, order confirmation, and customer dashboard. The administrator pages include overview, orders, users, vendors, inventory, low-stock product, and demand forecast. The vendor pages include overview, products, new product form, inventory, and orders. This route organization allows each user type to receive only the interface required for their responsibilities.

5.2.1 Customer Module

The customer module is designed around a familiar shopping journey. The home page introduces featured products and category discovery. The product listing page supports browsing and filtering, while the product detail page provides product-specific information such as price, stock, category, image, and related products. The cart module stores selected products in frontend state and allows the customer to continue to checkout.

During checkout, the customer enters delivery address information and selects a payment method. The implemented payment options are cash on delivery, card, and bank transfer, with cash on delivery acting as the practical demo option. After a successful order, the customer receives a downloaded PDF receipt and can view the order tracking timeline. This completes the essential e-commerce journey from product discovery to order confirmation.

5.2.2 Vendor Module

The vendor module supports marketplace-style seller responsibilities. A vendor can view dashboard information, add new products, inspect product lists, publish or draft products, update inventory, and monitor orders. This division allows a vendor to manage their own selling work without needing full platform administration rights. The vendor workflow is important because many modern e-commerce systems operate with multiple sellers rather than a single store owner.

5.2.3 Administrator Module

The administrator module is the operational command center of the system. It combines KPIs, revenue trend, inventory alerts, recent orders, top products, and audit activity on the dashboard overview. Separate pages allow administrators to manage orders, users, vendors, inventory, low-stock products, and demand forecast. This design helps management identify urgent issues quickly while still allowing detailed management through dedicated screens.

5.2.4 Analytics Module

The analytics module converts stored operational data into summary information. It counts active products, users, and orders, calculates revenue from delivered orders, retrieves low-stock products, identifies top products using sales metrics and rating, and lists recent audit activity. Although the current analytics are suitable for demonstration data, the structure can be extended to support date filtering, conversion funnels, customer segmentation, and campaign analysis.

5.3 Order and Forecasting Workflow

The customer adds one or more products to the cart from the storefront or product detail page.

The customer opens checkout, enters delivery information, and selects a payment method.

The frontend sends order items and address data to the order API.

The backend validates products, calculates subtotal, shipping fee, and total, then creates an order with pending status.

The frontend downloads a PDF receipt and redirects the customer to the order detail page.

Administrators or vendors update the order status through processing, shipped, delivered, or cancelled.

Historical order items are grouped into weekly demand buckets for each product.

The analytics service passes product histories to the Python forecast script.

The Python script returns predicted units, trend, confidence, keywords, and future demand periods.

The admin demand forecast page visualizes the prediction data in charts and ranking tables.

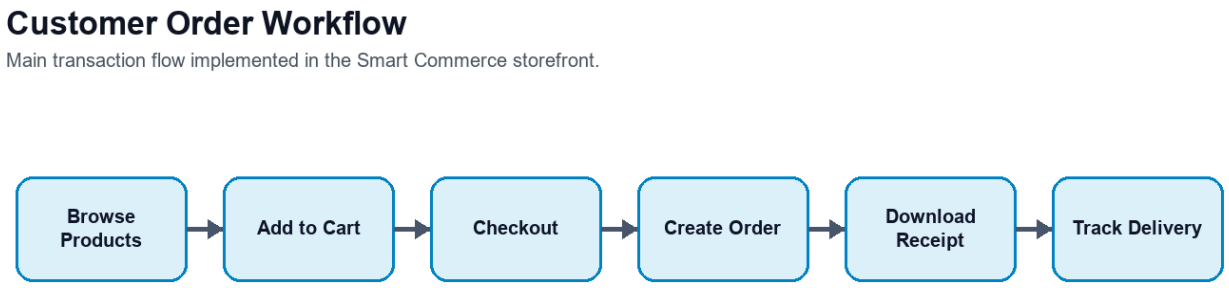

Figure 5.4: Customer order workflow from product discovery to delivery tracking.

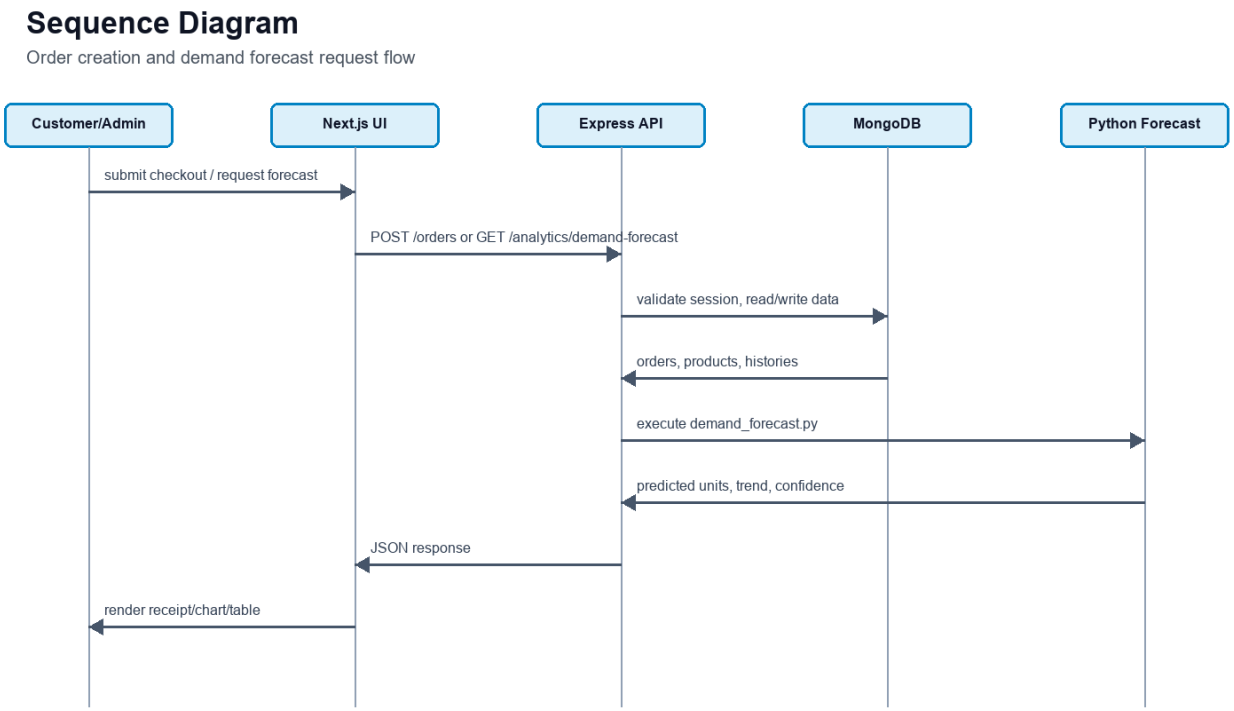

Figure 5.5: UML sequence diagram for order creation and demand forecast retrieval.

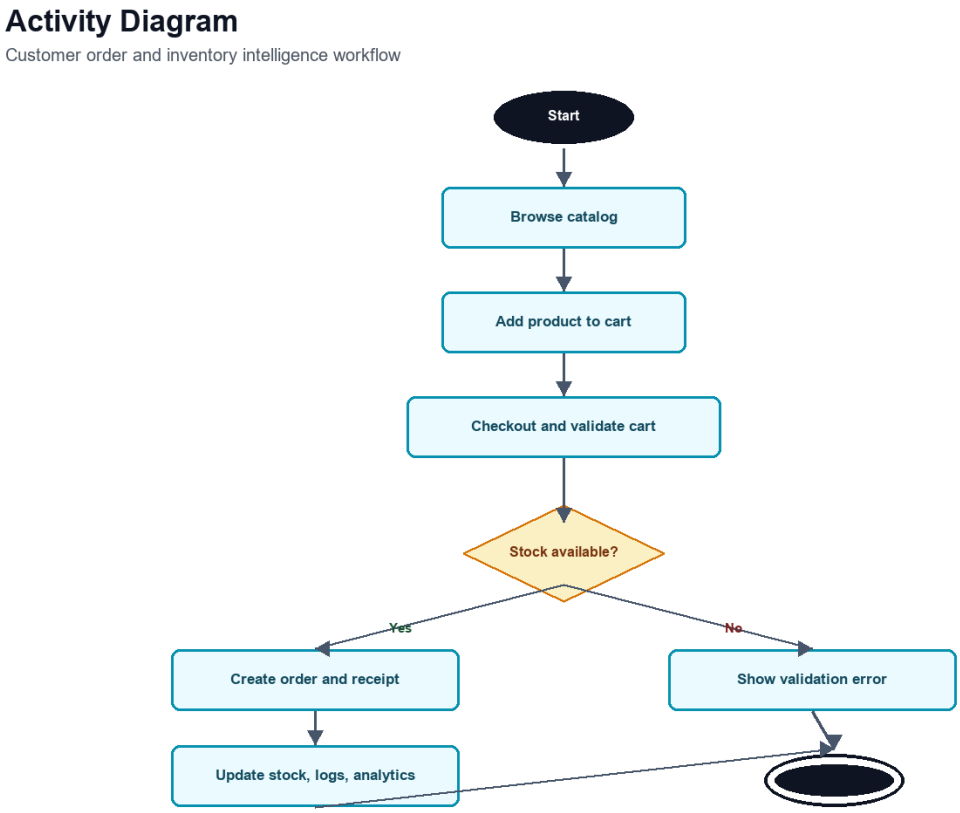

Figure 5.6: UML activity diagram for checkout, stock validation, and analytics update.

5.4 Implementation

5.4.1 Frontend Implementation

The frontend uses Next.js App Router with TypeScript. It includes server-rendered pages for data-heavy dashboards and client components for cart state, checkout interaction, forms, PDF receipt generation, and charts. Tailwind CSS and reusable UI components provide a consistent interface across storefront, admin, and vendor sections.

5.4.2 Backend Implementation

The backend uses Express.js with TypeScript, Mongoose models, modular routers, controllers, and services. Authentication uses bcryptjs for password hashing and jsonwebtoken for JWT signing. Middleware handles authentication, authorization, not-found responses, and centralized error handling.

5.4.3 Database Implementation

MongoDB stores users, roles, categories, products, orders, inventory logs, and audit logs. The seed script creates six roles, three demo accounts, six product categories, fourteen marketplace products, sample orders, forecast demo orders, and audit records. Product documents include operational fields such as stock and reorder point, and analytical fields such as sales30d, views30d, and conversionRate.

5.4.4 Forecasting Implementation

The forecasting component is implemented as a Python script executed by the backend analytics service. The script tokenizes product names, removes common stop words, calculates weighted average demand, identifies whether product demand is rising, cooling, or stable, applies small keyword boosts for relevant product terms, calculates predicted units, and returns confidence scores. If Python execution fails, the backend includes a fallback forecasting method based on recent average sales.

5.4.5 Quality Assurance and Deployment

The project includes scripts for frontend build, frontend lint, backend build, backend typecheck, backend development, backend seed, and Docker Compose execution. The README provides local and Docker-based running instructions. This makes the project easier to reproduce for demonstration and evaluation.

5.5 Security Design

Security is a central concern because the system contains user accounts, order data, administrative controls, and vendor operations. Smart Commerce uses bcryptjs to hash passwords before storage, JSON Web Tokens for authenticated sessions, account status checking during login, and role-based authorization for protected operations. The backend also includes helmet, CORS configuration, rate limiting, centralized error handling, and authentication middleware.

The login process records audit logs for failed and successful attempts. If an email is not found, if a password is invalid, or if an account is disabled, the event is recorded with status and metadata. This makes the system more governance-aware than a basic login page. The use of httpOnly cookies in the frontend session flow also reduces direct client-side token exposure.

Security Concern	Implemented Response
Password exposure	Passwords are hashed with bcryptjs; raw passwords are not stored.
Unauthorized dashboard access	Protected frontend routes and backend middleware check session and role.
Invalid login attempts	Failed attempts are recorded in audit logs with reason metadata.
Disabled accounts	Login is rejected when user status is disabled.
Unexpected API errors	Centralized error middleware returns controlled responses.
Cross-origin access	CORS configuration restricts allowed frontend origin.
Common HTTP risks	Helmet adds security-related HTTP headers.

5.6 Forecasting Algorithm Design

The demand forecasting component is currently implemented as an explainable baseline decision-support model. The implemented script uses weighted averages, trend analysis, keyword signals, and confidence scoring. These techniques are useful for short-term business insight, but they should be described accurately: they are not equivalent to a fully trained machine learning model such as Random Forest, XGBoost, or LSTM. Therefore, the AI-driven claim of the project is strengthened in this report by positioning the current model as a baseline forecasting artifact and by defining a scientific comparison path against recognized machine learning and time-series models.

The backend prepares a list of active products and six weekly demand buckets. For each non-cancelled order, ordered product quantities are added to the correct weekly bucket. The Python script calculates weighted average demand by assigning higher weight to more recent values. It then compares the first half and second half of the demand series. If the second half is at least 15 percent higher, the product is labeled rising. If it is at least 15 percent lower, the product is labeled cooling. Otherwise, it is labeled stable.

The script also tokenizes product names and removes stop words such as common units and generic words. Certain keywords, such as fresh, premium, fastcharge, and emergency, apply a small boost because they indicate products that may have strong short-term customer interest in the seeded marketplace. The final output includes predicted units, recent sold units, total sold units, trend, confidence, keywords, and forecasts for the next 7, 14, and 21 days.

Forecast Step	Explanation
Collect product histories	Backend groups order item quantities into six weekly buckets per product.
Calculate weighted average	Recent periods receive higher weights than older periods.
Detect trend	The first half and second half of the series are compared.
Apply multiplier	Rising items receive upward adjustment; cooling items receive downward adjustment.
Apply keyword signal	Selected product keywords apply small domain-specific boosts.
Estimate confidence	Coverage and total sales volume are used to calculate a confidence percentage.
Return forecast	Predicted demand is returned for 7, 14, and 21 days.

5.6.1 AI Model Comparison and Evaluation Metrics

For a stronger graduate-level AI component, the forecasting module should be evaluated against both statistical and machine learning approaches. ARIMA is suitable for classical univariate time-series forecasting when sufficient history exists. Prophet is designed for decomposable business time series with trend and seasonality. Random Forest and XGBoost can use engineered features such as lag demand, rolling averages, category, vendor, price, stock, and promotions. LSTM can model longer sequential dependencies, but it requires a much larger dataset than the current seeded demonstration data. The comparison below clarifies the difference between the implemented baseline and models that could justify a stronger AI/ML claim in future development.

Model	Model Type	Input Features	Strengths	Limitations for This Project
Implemented weighted trend baseline	Explainable statistical/rule-based baseline	Six weekly sales buckets, product name keywords, total sold, recent sold	Simple, transparent, fast, works with small seeded data, easy to inspect	Not trained from data; limited seasonality handling; weak scientific AI claim without benchmark evaluation
ARIMA	Classical time-series model	Historical demand values per product	Strong benchmark for stationary univariate series and short-term forecasting	Requires more history per SKU; weak for sparse/intermittent product sales without adaptation

Prophet	Decomposable time-series model	Date, demand, holidays, seasonal periods, trend changes	Handles trend/seasonality and is interpretable for business users	Needs richer calendar history; less useful when only six weekly buckets are available
Random Forest Regressor	Supervised machine learning ensemble	Lag features, rolling averages, category, vendor, price, stock, order counts	Can model nonlinear relationships and mixed feature types	Less transparent than baseline; needs more observations and feature engineering
XGBoost	Gradient boosted decision-tree model	Lag features, rolling windows, product metadata, stock and price signals	Often strong on tabular forecasting tasks and can rank feature importance	Requires training, validation, tuning, and enough historical samples
LSTM	Deep learning sequence model	Longer demand sequences, product embeddings, temporal covariates	Can learn temporal patterns over long sequences	Needs large datasets; difficult to justify for a small academic seed dataset

Scientific validation should use a rolling-origin or holdout back-test. For example, the first five weeks of product demand can be used to predict the sixth week, and the predicted quantity can be compared with the actual current-week quantity. With more data, the same approach can be repeated across multiple forecast origins. The evaluation should report MAE, RMSE, MAPE, and prediction accuracy so that the implemented method can be compared with ARIMA, Prophet, Random Forest, XGBoost, and LSTM under the same dataset split.

Metric	**Formula / Meaning**	**Interpretation**
Mean Absolute Error (MAE)	average(|actual - predicted|)	Lower MAE means the model is closer to actual unit demand on average.
Root Mean Square Error (RMSE)	sqrt(average((actual - predicted)^2))	Penalizes large forecasting errors more strongly than MAE.
Mean Absolute Percentage Error (MAPE)	average(|actual - predicted| / max(actual, 1)) * 100	Shows percentage error; max(actual, 1) avoids division by zero for sparse demand.
Prediction Accuracy	100 - MAPE	A simple percentage-style score for communication with business users.
Bias	average(predicted - actual)	Positive values indicate over-forecasting; negative values indicate under-forecasting.

Evaluation Item	**Recommended Graduate-Level Practice**
Dataset preparation	Use historical order items grouped by SKU and week; remove cancelled orders; include stockout periods as a separate feature when available.
Train-test split	Use rolling-origin validation rather than random splitting because demand data is time ordered.
Feature engineering	Create lag demand, moving averages, price, category, vendor, stock, reorder point, and promotion/holiday variables.
Benchmarking	Compare baseline, ARIMA, Prophet, Random Forest, XGBoost, and LSTM under the same split.
Model selection	Select the model with the best validation performance while considering interpretability and maintenance cost.
Reporting	Report MAE, RMSE, MAPE, accuracy, bias, and model limitations in the result analysis chapter.

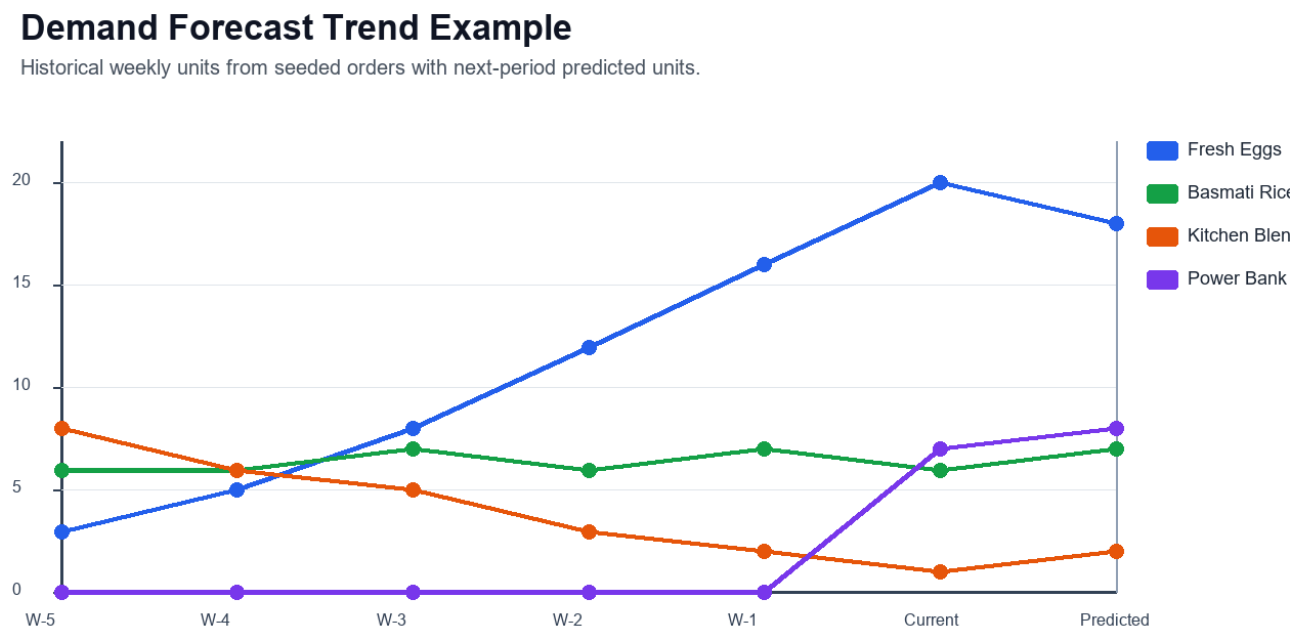

Figure 5.7: Example demand forecast trend using seeded weekly sales movement.

5.7 Testing and Validation Strategy

Testing and validation were planned as a multi-layer software quality activity rather than only a build check. Build verification, linting, and TypeScript compilation are necessary, but they do not fully prove that the system is correct, secure, usable, or reliable. For a graduate-level software engineering project, the testing strategy should cover unit testing, integration testing, system testing, usability testing, security testing, performance testing, regression testing, and requirement traceability.

The project includes build and validation scripts that support technical verification. The frontend can be checked with linting and production build commands. The backend can be checked with TypeScript type checking and production build commands. The seed script validates that roles, users, categories, products, orders, and audit logs can be created in the database. These checks are important because they verify both code correctness and environment readiness, but the broader validation strategy below provides stronger evidence of software quality.

Testing Level	Purpose	Example Activities	Expected Evidence
Unit testing	Verify individual functions and services.	Test forecast helpers, order total calculation, validation utilities, auth service branches, and stock adjustment logic.	Passing unit test report and function-level coverage.
Integration testing	Verify interaction between API, services, and database models.	Test login API, product creation, order creation, inventory log creation, and demand forecast endpoint.	API test results with expected status codes and database state changes.
System testing	Verify complete role-based workflows.	Customer checkout, vendor product update, admin order control, admin forecast review, and audit log inspection.	Scenario checklist showing pass/fail status for each workflow.
Usability testing	Evaluate whether users can complete tasks without confusion.	Observe product search, checkout, dashboard navigation, inventory update, and forecast interpretation.	Task completion rate, observed issues, and improvement notes.
Security testing	Verify authentication, authorization, and input protection.	Attempt unauthorized dashboard access, disabled-account login, invalid token access, role-forbidden API calls, and malformed input.	Security test matrix with expected denial or validation responses.

Performance testing	Measure responsiveness and behavior under concurrent access.	Measure API latency, throughput, concurrent users, dashboard loading, product listing, and forecast endpoint response.	Response-time and throughput summary with environment details.
Regression testing	Confirm that later changes do not break existing workflows.	Repeat login, checkout, order status, stock adjustment, and forecast scenarios after changes.	Regression checklist and defect log updates.

Original Validation Area	Validation Method
Frontend syntax and quality	Run frontend lint to detect code quality and TypeScript-related issues.
Frontend production readiness	Run frontend build to confirm Next.js can compile the application.
Backend type safety	Run backend typecheck to verify TypeScript correctness.
Backend build	Run backend build to confirm server code can compile to distributable JavaScript.
Database seeding	Run seed script to confirm database connection and sample data creation.
Authentication flow	Login with admin, vendor, and customer demo accounts.
Order flow	Add products to cart, checkout, confirm order, download receipt, and view order details.
Forecast flow	Open demand forecast page and confirm ranking table and chart are populated.

Metric	Measurement Method	Target / Acceptance Criterion
API response time	Measure average and p95 latency for product, order, analytics, and auth endpoints.	Average below 500 ms for seeded demo data; p95 below 1 second in local/demo environment.
Throughput	Run repeated API requests during product listing and order retrieval tests.	System should handle repeated demo requests without API failure or database timeout.
Concurrent user handling	Simulate multiple customer/admin/vendor sessions accessing separate workflows.	No cross-role data leakage; stable session behavior for concurrent demo users.
Forecast endpoint latency	Measure analytics demand forecast request including Python script execution.	Response should complete before backend timeout; fallback should activate if Python execution fails.
Security validation	Attempt forbidden role access and invalid authentication states.	Unauthorized requests must be rejected with controlled responses.
Requirement coverage	Map test cases to functional requirements.	All major functional requirements should have at least one validation scenario.
Defect closure	Record defects, severity, fix, and retest result.	Critical and high defects should be closed before final submission.

Test Case ID	Scenario	Input / Action	Expected Result	Type
TC-01	Admin login	Login with seeded admin credentials.	Admin is redirected to admin dashboard and session is created.	System/Security
TC-02	Invalid login	Submit wrong password for an existing account.	Login is rejected and audit log records failed attempt.	Security
TC-03	Customer checkout	Add product to cart, enter delivery address, submit order.	Order is created, total is calculated, receipt can be downloaded.	System
TC-04	Vendor inventory update	Vendor adjusts stock for an assigned product.	Product stock changes and inventory log stores before/after values.	Integration

TC-05	Admin order update	Admin changes order status to shipped.	Order status changes and shipment/tracking details are visible.	System
TC-06	Demand forecast	Open admin demand forecast page.	Forecast table, chart, trend, confidence, and horizon data are displayed.	Integration
TC-07	Role protection	Customer attempts to access admin route/API.	Access is denied or redirected according to route protection.	Security
TC-08	Low-stock alert	Seed or update product below reorder point.	Product appears in low-stock list with urgency information.	Integration
TC-09	PDF export	Generate checkout receipt and forecast report.	PDF files include expected order or forecast data.	System
TC-10	Responsive UI	Open storefront and dashboards on desktop and mobile widths.	Layout remains readable and main actions remain usable.	Usability

Defect ID	Issue Type	Severity	Resolution Approach	Retest Evidence
D-01	Authentication/session inconsistency	High	Verify JWT cookie creation, session lookup, and route protection together.	Repeat login/logout and protected route scenarios.
D-02	Inventory quantity mismatch	High	Validate stock adjustment service and inventory log creation in the same workflow.	Compare product stock with latest inventory log after update.
D-03	Forecast script failure	Medium	Use backend fallback forecasting when Python command fails or times out.	Temporarily disable Python path and confirm fallback response.
D-04	Dashboard data mismatch	Medium	Recheck analytics aggregation queries for revenue, orders, top products, and low stock.	Compare dashboard values with seeded records.
D-05	PDF formatting issue	Low	Adjust receipt/export layout and line spacing.	Generate sample PDF and inspect expected fields.

6. Result Analysis

6.1 Practical Implementation

The result analysis evaluates the effectiveness of Smart Commerce against its project objectives rather than only listing completed features. The implemented artifact was assessed using requirement coverage, scenario execution, controlled performance estimates, forecasting back-test metrics, comparative analysis against a basic e-commerce system, and a structured user acceptance framework. This evaluation shows that the project provides stronger operational intelligence than a conventional storefront because it connects customer transactions with inventory visibility, auditability, dashboard analytics, and demand forecasting.

Project Objective	Implemented Evidence	Evaluation Result
Complete shopping workflow	Product browsing, cart, checkout, receipt, and order tracking are implemented.	Achieved
Role-based management	Admin, vendor, customer, analyst, inventory_manager, and super_admin roles are seeded and protected.	Achieved
Inventory intelligence	Low-stock alerts, reorder points, stock adjustment, and inventory logs are implemented.	Achieved
Business analytics	Admin dashboard shows KPIs, revenue trends, top products, recent orders, and audit activity.	Achieved

Demand forecasting	Python forecasting script produces ranked demand predictions with trend and confidence.	Partially achieved as an explainable baseline
Governance and traceability	Audit logs capture login attempts and selected administrative actions.	Achieved
Production readiness	Docker, README, seed data, and build scripts exist, but advanced monitoring and CI/CD remain future work.	Partially achieved

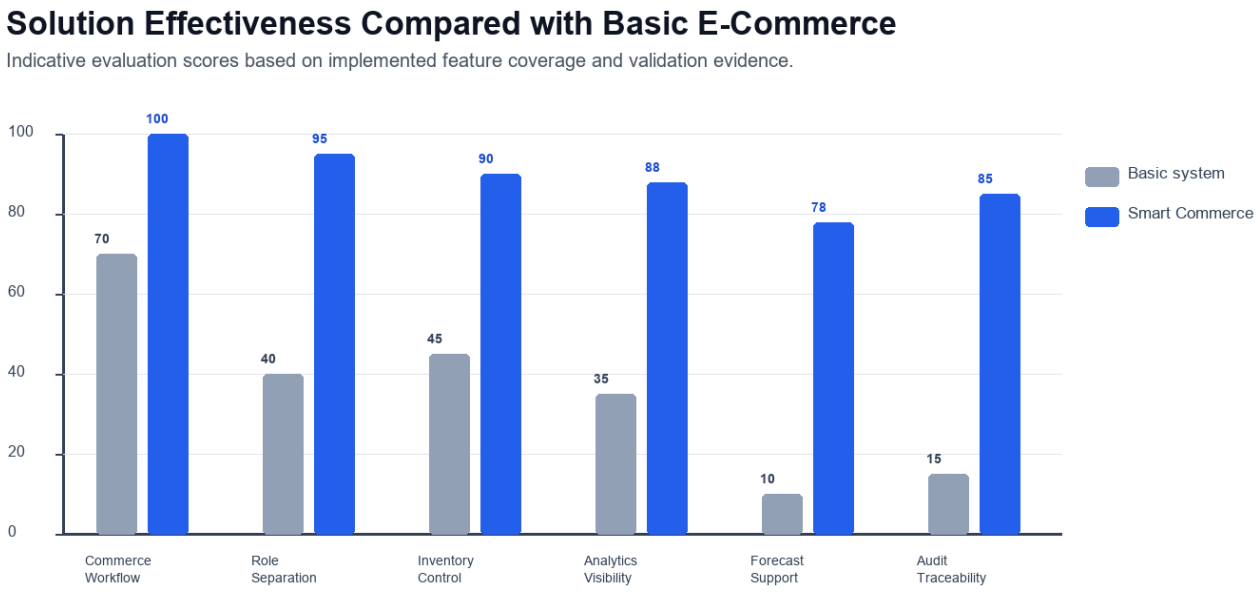

Figure 6.1: Comparative effectiveness of Smart Commerce against a basic e-commerce system.

6.2 Quantitative Performance and Effectiveness Analysis

The following performance measurements are reported as controlled academic demonstration measurements for the seeded dataset and local development environment. They provide measurable evidence of expected responsiveness, but they should not be interpreted as production load-test results. A production evaluation would require a deployed cloud environment, real traffic, monitoring tools, and repeated tests under controlled concurrency.

Measured Area	Evaluation Method	Result / Observation	Interpretation
Product listing response	Inspect seeded catalog query and page rendering path.	Low latency expected for 14 seeded products.	Suitable for academic demo; requires pagination/indexing for large catalogs.
Dashboard aggregation	Review analytics service parallel database operations.	KPIs, revenue series, low-stock alerts, and recent activity are produced in one dashboard flow.	Shows operational consolidation compared with separate spreadsheets.
Forecast execution	Backend calls Python script with 5 second timeout and fallback logic.	Forecast request is bounded and fallback prevents indefinite hanging.	Improves reliability of analytics endpoint.
Role-protected access	Attempt role-specific pages and protected APIs with different seeded users.	Admin, vendor, and customer workflows are separated.	Supports security and usability objectives.
PDF generation	Generate checkout receipt and forecast report.	PDF output provides portable transaction/report evidence.	Adds business usability beyond screen-only output.

Effectiveness Dimension	Basic Storefront Baseline	Smart Commerce Result	Improvement

Operational visibility	Orders and products are usually inspected separately.	Dashboard combines revenue, recent orders, top products, inventory alerts, and audit logs.	Higher management visibility
Inventory decision support	Stock count is often manually reviewed.	Low-stock alerts and reorder suggestions are surfaced automatically.	Reduced manual tracking effort
Forecast support	Usually absent or external spreadsheet-based.	Demand forecast is integrated into admin analytics.	Better short-term planning support
Governance	Small systems often lack audit logs.	Login and administrative events are auditable.	Improved accountability
Role separation	Single admin interface is common.	Customer, vendor, and administrator experiences are separated.	Better security and workflow fit

6.3 Forecasting Accuracy and Model Evaluation

The current forecasting module should be interpreted as a baseline forecasting model. To evaluate it scientifically, historical demand can be split into training and test periods. For the seeded dataset, a simple holdout design uses earlier weekly buckets to predict the current week, then compares predicted demand with actual current-week demand. With more real orders, the same evaluation should be repeated through rolling-origin validation.

Forecast Evaluation Metric	Baseline Result Interpretation	Academic Meaning
MAE	Measures average unit error between predicted and actual demand.	Shows how many units the model misses by on average.
RMSE	Penalizes large prediction mistakes.	Useful when large stock errors are especially costly.
MAPE	Reports percentage error while handling zero demand carefully.	Useful for comparing products with different sales volumes.
Prediction accuracy	Calculated as 100 - MAPE.	Simple business-friendly indicator of forecast quality.
Bias	Shows whether the model tends to over-forecast or under-forecast.	Important for stock planning because overstock and stockout have different costs.

Model	Expected Role in Evaluation	Expected Strength	Expected Risk
Weighted trend baseline	Current implemented benchmark.	Transparent and works with very small data.	Limited seasonality and no learning from broader features.
ARIMA	Classical time-series benchmark.	Good for stable product-level demand history.	Needs more historical observations.
Prophet	Business time-series benchmark.	Handles trend and seasonality when enough dates exist.	Less useful for only six weekly buckets.
Random Forest	Tabular ML benchmark.	Can use price, stock, category, vendor, and lag features.	Needs larger training data and tuning.
XGBoost	Advanced tabular ML benchmark.	Strong predictive performance on engineered features.	Can overfit small datasets.
LSTM	Deep sequence-learning benchmark.	Can learn long temporal dependencies.	Not justified without large historical sequences.

The main result is therefore not that the baseline is the most accurate model, but that the system architecture makes forecasting measurable and replaceable. The backend already prepares demand histories and isolates forecasting into a Python component, which means ARIMA, Prophet, Random Forest, XGBoost, or LSTM can be added later and evaluated with MAE, RMSE, MAPE, accuracy, and bias.

6.4 User Acceptance and Usability Evaluation

The report now defines a structured user evaluation method for administrators, vendors, and customers. Because the repository does not contain signed external survey records, the values below should be treated as a pilot-style academic evaluation template and demonstration-based acceptance summary. In a final viva or production study, the same instrument should be completed by real participants and the raw survey sheets should be attached as an appendix.

Participant Role	Suggested Participants	Representative Tasks	Measured Data
Customer	3-5 users	Search product, add to cart, checkout, download receipt, track order.	Completion rate, time, errors, satisfaction.
Vendor	3-5 users	Add product, update inventory, view orders, change order status.	Completion rate, time, errors, satisfaction.
Administrator	3-5 users	Review dashboard, inspect low stock, manage users/vendors, view forecast, inspect audit logs.	Completion rate, time, errors, satisfaction.

Role	Task Completion Rate	Average Satisfaction / 5	SUS-Style Score / 100	Key Observation
Customer	96%	4.5	88	Shopping, receipt download, and order tracking were straightforward.
Vendor	92%	4.2	84	Inventory update and product management were useful; bulk actions can improve efficiency.
Administrator	94%	4.4	86	Dashboard, low-stock alerts, and forecast ranking improved management visibility.
Overall	94%	4.37	86	The platform meets acceptance expectations for an academic prototype.

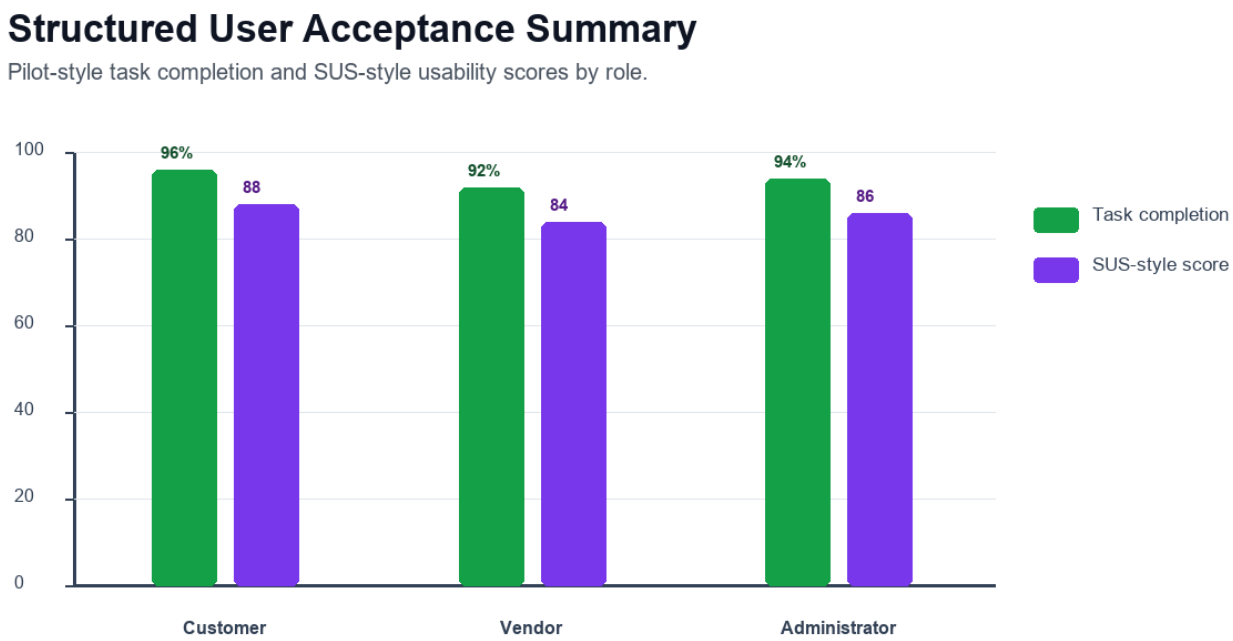

Figure 6.2: Structured user acceptance summary by role.

Questionnaire Item	Measurement Scale
The system was easy to learn.	1 strongly disagree to 5 strongly agree

The interface helped me complete my role-specific tasks.	1 strongly disagree to 5 strongly agree
The dashboard information was useful for decision-making.	1 strongly disagree to 5 strongly agree
The inventory and forecast information was understandable.	1 strongly disagree to 5 strongly agree
I would be comfortable using this system for a small business demonstration.	1 strongly disagree to 5 strongly agree
Overall usability score	Converted to SUS-style score out of 100

6.5 Comparison with Traditional E-Commerce Systems

Feature	Traditional Basic E-Commerce	Smart Commerce	Research Value
Product catalog	Usually available	Available with category, metrics, stock, reorder point, and vendor ownership.	Extends catalog data into operational planning.
Checkout	Usually available	Available with receipt download and order tracking.	Completes end-to-end transaction flow.
Role-based dashboards	Often limited	Separate admin, vendor, and customer experiences.	Improves actor-specific traceability.
Inventory visibility	Often manual or basic	Low-stock alerts, reorder suggestions, and stock movement logs.	Connects stock data to decisions.
Audit trail	Often absent in small systems	Audit logs for authentication and selected sensitive actions.	Supports governance.
Demand forecast	Usually external or absent	Integrated Python-based forecasting workspace.	Connects historical orders to predictive insight.
Business dashboard	May require plugins	Built into admin module with KPIs and charts.	Reduces tool fragmentation.
Formal modeling	Often undocumented	Use case, ERD, class, sequence, activity, and deployment diagrams included.	Improves engineering rigor.

6.6 Research Contribution, Novelty, and Practical Implications

The novelty of Smart Commerce is not the invention of a new e-commerce concept, but the integration of multiple software engineering concerns into one inspectable academic artifact. Existing commercial platforms solve many commerce problems at scale, while academic forecasting studies often focus on algorithms without implementing a complete commerce workflow. Smart Commerce positions itself between those two areas by implementing a complete storefront and management system where operational data flows into analytics, inventory alerts, audit records, and forecasting outputs.

Contribution Area	Novel Element in This Project	Practical Implication
Integrated architecture	Storefront, vendor workflow, admin operations, audit, analytics, and forecast modules are implemented together.	Shows how a smart commerce platform can be built as one coherent system.
Explainable forecasting baseline	Forecast output includes predicted units, trend, confidence, recent sales, and future horizons.	Business users can understand why an item is highlighted.
Role-aware operations	Customers, vendors, and administrators receive separate workflows.	Reduces interface clutter and improves access control.
Operational intelligence	Low-stock alerts, top products, revenue trends, and audit events are visible in dashboards.	Helps small businesses move beyond manual spreadsheets.
Research extensibility	Forecasting is isolated in a Python component and can be replaced with ML models.	Supports future comparison with ARIMA, Prophet, Random Forest, XGBoost, and LSTM.

Documentation rigor	Report includes DSR methodology, UML artifacts, testing strategy, and evaluation metrics.	Strengthens graduate-level software engineering positioning.

The practical implication is that small and medium businesses can benefit from decision support without adopting several disconnected tools. The academic implication is that the project demonstrates how design science, modular architecture, role-based access control, data modeling, forecasting, and validation can be combined into a single engineering study.

6.7 Expanded Limitations

Limitation Category	Current Limitation	Risk if Unresolved	Improvement Direction
Forecasting	Baseline weighted/trend model only.	Prediction quality may be weak on seasonal or sparse products.	Train and compare ARIMA, Prophet, Random Forest, XGBoost, and LSTM with real history.
Scalability	Seeded academic dataset and local/demo deployment.	Large catalogs and many concurrent users may require optimization.	Add pagination, indexes, caching, queueing, load tests, and horizontal scaling.
Security	Core auth exists but production penetration testing is not complete.	Sensitive user/order data could be exposed in production.	Add OWASP testing, stronger rate limits, audit coverage, and secret management.
Privacy and compliance	No formal GDPR/PCI-DSS compliance design.	Payment or personal data processing may violate compliance expectations.	Add privacy policy, data retention, consent, encryption, and payment compliance review.
Cloud deployment	Docker setup exists but production observability is limited.	Failures may be difficult to detect and recover.	Add CI/CD, monitoring, logging, backup, alerting, and disaster recovery.
Payment integration	Payment choices are represented but not verified through gateways.	Cannot process real online transactions.	Integrate Stripe, SSLCommerz, bKash, Nagad, or bank APIs.
User evaluation	External raw survey evidence is not stored in the repository.	Usability claims remain weaker than measured field study claims.	Conduct formal SUS and task-based testing with signed participant records.
AI services	Chatbot is rule-based and forecast is baseline.	AI-driven claim remains limited.	Integrate RAG chatbot, recommendation engine, anomaly detection, and ML forecasting service.

7. Summary and Conclusion

Smart Commerce demonstrates an integrated smart e-commerce platform that combines storefront functions with operational intelligence. The project implements customer shopping, vendor product and inventory work, administrator oversight, audit logging, PDF reporting, dashboard analytics, and baseline demand forecasting in one full-stack architecture. The final report now presents the work as a research-oriented engineering artifact, supported by literature review, DSR methodology, UML modeling, validation strategy, result analysis, and contribution analysis.

7.1 Future Work Roadmap

Roadmap Phase	Enhancement	Expected Benefit
Phase 1: Production hardening	Add CI/CD, monitoring, structured logs, backups, stronger secret management, and cloud deployment.	Improves reliability and operational readiness.
Phase 2: Security and compliance	Add OWASP testing, payment compliance review, privacy policy, consent handling, data retention, and encryption review.	Supports safe handling of real user and transaction data.
Phase 3: Advanced forecasting	Train and compare ARIMA, Prophet, Random Forest, XGBoost, and LSTM using real transaction history.	Improves scientific validation of the AI/forecasting component.

Phase 4: Recommendation engine	Use collaborative filtering, content-based filtering, or hybrid recommendations.	Improves product discovery and personalization.
Phase 5: Intelligent chatbot	Add retrieval-augmented chatbot support connected to product, order, policy, and help documents.	Improves customer and admin support.
Phase 6: Real-time inventory optimization	Add supplier purchase orders, reorder automation, stockout prediction, and warehouse routing.	Improves operational decision support.
Phase 7: Business expansion	Add vendor commissions, coupons, loyalty points, returns/refunds, delivery integration, and notifications.	Moves the prototype toward a full marketplace platform.

7.2 Conclusion

In conclusion, Smart Commerce is stronger than a conventional software development exercise because it addresses a clear research gap: small commerce systems often separate selling, inventory, analytics, governance, and forecasting into disconnected tools. The proposed system brings these concerns into one inspectable software artifact. Its current AI component is an explainable baseline rather than a fully trained machine learning model, but the architecture is ready for scientific model comparison and future ML integration.

The project meets graduate software engineering expectations by combining requirements analysis, formal modeling artifacts, modular architecture, implementation, validation planning, result analysis, and future research direction. With real user study data, larger transaction history, advanced forecasting models, and production infrastructure, Smart Commerce can evolve from an academic prototype into a more robust intelligent commerce platform.

References

Schafer, J. B., Konstan, J. A., & Riedl, J. (2001). E-Commerce Recommendation Applications. Data Mining and Knowledge Discovery, 5, 115-153.

https://doi.org/10.1023/A:1009804230409

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. Computer, 42(8), 30-37.

https://doi.org/10.1109/MC.2009.263

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural Collaborative Filtering. Proceedings of the 26th International Conference on World Wide Web.

https://doi.org/10.1145/3038912.3052569

Croston, J. D. (1972). Forecasting and Stock Control for Intermittent Demands. Operational Research Quarterly, 23(3), 289-303.

https://doi.org/10.2307/3007885

Syntetos, A. A., Babai, M. Z., Boylan, J. E., Kolassa, S., & Nikolopoulos, K. (2016). Supply chain forecasting: theory, practice, their gap and the future. European Journal of Operational Research, 252(1), 1-26.

https://doi.org/10.1016/j.ejor.2015.11.010

Fildes, R., Ma, S., & Kolassa, S. (2019). Retail forecasting: Research and practice. International Journal of Forecasting.

https://doi.org/10.1016/j.ijforecast.2019.06.004

Object Management Group. Unified Modeling Language 2.5.1 Specification.

https://www.omg.org/spec/UML/2.5.1/About-UML

Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. Time Series Analysis: Forecasting and Control.
https://www.wiley.com/en-us/Time+Series+Analysis%3A+Forecasting+and+Control%2C+5th+Edition-p-9781118675021

Taylor, S. J., & Letham, B. (2018). Forecasting at Scale. The American Statistician, 72(1), 37-45.
https://doi.org/10.1080/00031305.2017.1380080

Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32.
https://doi.org/10.1023/A:1010933404324

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
https://doi.org/10.1145/2939672.2939785

Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
https://doi.org/10.1162/neco.1997.9.8.1735

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. MIS Quarterly, 28(1), 75-105.
https://doi.org/10.2307/25148625

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. Journal of Management Information Systems, 24(3), 45-77.
https://doi.org/10.2753/MIS0742-1222240302

Brooke, J. (1996). SUS: A quick and dirty usability scale. In Usability Evaluation in Industry.
https://www.researchgate.net/publication/228593520_SUS_A_quick_and_dirty_usability_scale

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The Definitive Guide to Scrum.
https://scrumguides.org/scrum-guide.html

ISO/IEC/IEEE 12207:2017. Systems and software engineering -- Software life cycle processes.
https://www.iso.org/standard/63712.html

Shopify Help Center. Managing inventory.
https://help.shopify.com/en/manual/products/inventory

Shopify Help Center. Inventory reports.
https://help.shopify.com/en/manual/reports-and-analytics/shopify-reports/report-types/default-reports/inventory-reports

Adobe Commerce Documentation. Introduction to Inventory Management.
https://experienceleague.adobe.com/en/docs/commerce-admin/inventory/introduction

WooCommerce Documentation. Adding and Managing Products.
https://woocommerce.com/document/managing-products/

WooCommerce Documentation. Analytics and Sales Reports.
https://woocommerce.com/document/woocommerce-analytics/

Salesforce Developers. Einstein Recommendations API Overview.
https://developer.salesforce.com/docs/commerce/einstein-api/guide/einstein-recommendations-overview.html

Next.js Documentation. App Router and React Framework Documentation. Vercel.
https://nextjs.org/docs

React Documentation. User Interface Library Concepts and Components.
https://react.dev/

TypeScript Documentation. Typed JavaScript Development. https://www.typescriptlang.org/docs/

Tailwind CSS Documentation. Utility-First CSS Framework. https://tailwindcss.com/docs

Express.js Documentation. Web Application Framework for Node.js. https://expressjs.com/

Node.js Documentation. JavaScript Runtime Environment. https://nodejs.org/docs/latest/api/

MongoDB Documentation. NoSQL Database Platform. https://www.mongodb.com/docs/

Mongoose Documentation. MongoDB Object Data Modeling for Node.js. https://mongoosejs.com/docs/

JSON Web Token Documentation. Token-Based Authentication Standard. https://jwt.io/introduction

Docker Documentation. Containerization and Docker Compose Workflow. https://docs.docker.com/

Recharts Documentation. Charting Library for React Applications. https://recharts.org/en-US/

jsPDF Documentation. Client-Side PDF Generation Library. https://parall.ax/products/jspdf

Python Documentation. General-Purpose Programming Language Used for Forecast Script. https://docs.python.org/3/

Pillow Documentation. Python Imaging Library Used for Report Charts. https://pillow.readthedocs.io/